API mocking with WireMock: interview questions, answered with this project

Questions about mocking third-party APIs that come up in SDET and QA lead interviews. Where the project shows the idea, the answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/wiremock-api-mocking.

Versions: Java 21, WireMock 3.13, JUnit 6, AssertJ 3, Jackson 2.

Contents

1. Mocking basics (Q1-Q14)
2. Matching and responses (Q15-Q28)
3. Failures, retries and state (Q29-Q42)
4. Verification and test design (Q43-Q52)
5. Process and scenarios (Q53-Q60)

1. Mocking basics

Q1. What is WireMock?

An HTTP server you control from tests. You tell it which requests to expect and how to answer, and it records what it received. Your code talks to it over real HTTP, as it would to the real API.

In the project: two WireMock servers play the payment provider and the shipping carrier.

Q2. Why not just use the provider's sandbox?

A sandbox is useful, but it's shared, can be down or slow, and has rate limits. Above all, it won't reliably produce the failures you need: a dropped connection, a `429` on the second attempt, a broken body. Those paths are where integrations go wrong.

Q3. What is the difference between a stub, a mock and a fake?

A stub gives prepared answers. A mock also checks how it was called. A fake is a simple working version, such as an in-memory database. WireMock can act as a stub and as a mock, because it answers and also verifies requests.

Q4. WireMock or Mockito?

Mockito replaces a Java object inside the process, so no HTTP happens. WireMock keeps the real HTTP client, so headers, URL encoding, JSON, timeouts and status handling are all tested. Mocking your own HTTP client class leaves exactly that code untested.

In the project: `TestClients` builds the real clients; only the base URL differs from production.

Q5. What is service virtualization?

Simulating a dependency so you can test without it. WireMock is a lightweight form of it. The term is often used for larger, shared simulated environments, but the idea is the same.

Q6. Where does WireMock sit in the test pyramid?

Between unit and end-to-end tests: integration tests of one component and its HTTP boundary. They're nearly as fast as unit tests but check real I/O code.

Q7. WireMock or contract testing with Pact?

They answer different questions. WireMock asks "does my code cope with every answer?" Pact asks "does the real provider still give the answer I expect?" A mock can't notice the provider changing, which is the gap Pact fills. I'd use both.

In the project: the README and design document name this limit and link the Pact repository.

Q8. WireMock or Testcontainers?

Testcontainers runs the real thing in Docker, which suits what you own or can run: a database, a broker. A third-party SaaS API can't run in a container, so you mock it. The two also combine: a WireMock container can stand in for an API.

Q9. What are the ways to run WireMock?

Inside the tests through the JUnit extension, as a standalone JAR, as a Docker image, or hosted. Inside the tests is simplest. Standalone suits a shared stub for several teams, or for a UI or mobile app under test.

In the project: inside the tests, through `WireMockExtension`.

Q10. How does your code reach WireMock instead of the real API?

The base URL must be configurable. Point it at `http://localhost:<port>` in tests. If a URL is hard-coded, that is the first thing to fix, and it's a useful testability finding on its own.

In the project: `ApiConfig` holds the base URL.

Q11. Why use a dynamic port?

A fixed port clashes with other builds and leftover processes. With `dynamicPort()` the operating system picks a free port and the test reads it from the server.

In the project: `MockServers` uses `dynamicPort()` for every server.

Q12. Why one server per API rather than one for all?

A stub for one API can't then answer a call meant for the other, so a client using the wrong base URL fails. It also matches production, where they're different hosts.

In the project: `MockedApisTest` has `PAYMENT_API` and `SHIPPING_API`.

Q13. Should a server start per test or per class?

Per class, in most cases: starting one is much slower than resetting one. WireMock's JUnit extension resets the stubs and request log before each test, so tests stay independent.

In the project: the extensions are static fields.

Q14. What does a mock not prove?

That the real API behaves like your stub. It also doesn't cover TLS, proxies, DNS, or real network timing. You still need a small number of checks against the real sandbox, and contract tests.

In the project: see "Limits" in `docs/framework-design.md`.

2. Matching and responses

Q15. What can a stub match on?

Method, URL, path, query parameters, headers, cookies, body, and basic auth. Matchers include `equalTo`, `containing`, `matching` (regular expression), `equalToJson`, `matchingJsonPath` and `equalToXml`.

Q16. What is the difference between `urlEqualTo` and `urlPathEqualTo`?

`urlEqualTo` compares the path and query string together. `urlPathEqualTo` compares the path only, so query parameters can be matched one by one, in any order.

In the project: `ShippingCarrierStub.rates()` uses `urlPathEqualTo` and `withQueryParam`.

Q17. How strict should a stub be?

As strict as the real API, for what matters. If the provider needs an API key, the stub should need it too; otherwise you can delete the header and stay green. Don't match on details that don't matter, or tests break for no reason.

In the project: `PaymentProviderStub.charge()` needs the three headers the provider needs.

Q18. How does `equalToJson` work?

It compares JSON, not text, so spacing and field order don't matter. Options let you ignore array order or extra elements. By default, a missing or extra field fails.

In the project: `tc102` uses it to pin the exact body.

Q19. When is `matchingJsonPath` better than `equalToJson`?

When only part of the body matters, for example one field, and the rest may change between tests.

In the project: `payment-declined.json` matches `cardToken` only.

Q20. How does stub priority work?

The lower number wins, and the default is 5. With equal priority, the most recently added stub wins. Use a general stub at a high number and specific ones at a low number.

In the project: `tc162`: the decline at priority 1 beats the capture at priority 5.

Q21. What is response templating?

The response is built from the request with Handlebars, for example `{{jsonPath request.body '$.orderId'}}`. It makes a stub behave like an API, and it proves the answer came from this request.

In the project: `CAPTURED_BODY` builds `pay_ORD-7` from the order id (`tc111`).

Q22. Local or global templating?

Local applies to stubs that name the `response-template` transformer. Global applies to all stubs. Local is safer, because a body that happens to contain `{{` isn't processed by surprise.

In the project: local, through `withTransformers("response-template")`.

Q23. Where do large response bodies go?

In the `__files` folder, referenced with `bodyFileName`. The stub stays readable and the body can be copied from a real response.

In the project: `shipping-rate.json` (`tc161`).

Q24. Stubs in Java or in JSON files?

Java is good for behaviour that varies per test, and it gets IDE support. JSON suits stable stubs, non-developers, and sharing with standalone WireMock.

In the project: both, with Java in `stubs/` and JSON in `src/test/resources/wiremock`.

Q25. How do you stub a redirect or a binary response?

A redirect is a `3xx` status with a `Location` header (`temporaryRedirect` is a shortcut). Binary content uses `withBody(byte[])` or a file in `__files`.

Q26. What is the admin API?

A REST API on the same port under `/__admin`: add stubs, list requests, see unmatched ones, reset, start a recording. It's how you control a standalone WireMock, for example from a non-Java test.

Q27. Can WireMock call back, like a webhook?

Yes. A stub can have a webhook listener that sends a request after answering, with an optional delay. That suits providers that confirm a payment later.

Q28. How do you extend WireMock?

With extensions: response transformers, request filters, custom matchers, and serve event listeners. I'd try templating first, since most needs don't require custom code.

3. Failures, retries and state

Q29. Which network faults can WireMock inject?

`EMPTY_RESPONSE`, `MALFORMED_RESPONSE_CHUNK`, `RANDOM_DATA_THEN_CLOSE` and `CONNECTION_RESET_BY_PEER`.

In the project: `@EnumSource(Fault.class)` runs all of them in `tc131`, `tc132` and `tc144`.

Q30. Which delays can it simulate?

A fixed delay, random delays (uniform or lognormal), and a chunked dribble that sends the body slowly. The fixed delay is best for a deterministic timeout test.

In the project: `withFixedDelay` in `tc133`, `tc134` and `tc146`.

Q31. What are stateful scenarios?

A small state machine: each stub applies in one state and can move to the next. The same request can then get a different answer each time.

In the project: `failThenCapture` builds `503`, `503`, `201` for `tc121`.

Q32. How do you test a retry?

Fail first, succeed later, then assert three things: the final result, the number of requests, and that the requests were safe to repeat. A test that checks only the result passes when no retry happens at all, if the stub always succeeds.

In the project: `tc121` and `tc122`.

Q33. What is an idempotency key?

A unique value per operation, sent with the request. If the provider sees it twice, it returns the first result and doesn't act again. That makes a retry safe for operations like payments.

In the project: the order number, identical on every attempt (`tc122`, `tc156`).

Q34. Why is a payment timeout not a failed payment?

Because the client doesn't know. The provider may have taken the money and the answer got lost. Reporting "failed" invites the customer to pay twice, so the honest state is "unknown", resolved by a later check.

In the project: `PAYMENT_UNKNOWN` (`tc131`, `tc133`, `tc154`).

Q35. Which status codes should be retried?

`429` and `5xx`, plus network errors, when the operation is idempotent. Not other `4xx` answers, because the same request gets the same refusal.

In the project: `RETRYABLE` in `PaymentClient`; `tc114` proves the others aren't retried.

Q36. How do you test `Retry-After`?

Stub a `429` with the header and assert the wait. Include the awkward cases: a huge value, and the header's date form.

In the project: `tc125`, `tc126`, `tc127`.

Q37. How do you test waits without slowing the suite?

Make the wait replaceable. An injected interface records the duration in tests, so the test is fast and the value can be asserted, which real sleeping can't do.

In the project: `Sleeper` and `RecordingSleeper`.

Q38. What is exponential backoff, and why add jitter?

Each wait doubles, which gives a struggling service room. Jitter adds randomness so that many clients don't all retry at the same moment. This project has no jitter, because its waits are asserted exactly; a production client with many instances should add it.

In the project: `RetryPolicy.delayAfter` (`tc124`).

Q39. What is a circuit breaker, and how would you test it?

It stops calling a failing service for a while. To test it: make the stub fail enough times to open the circuit, check that no further requests arrive, then make the stub healthy and check that calls resume. The verification of "no requests" is what WireMock adds.

Q40. How do you test graceful degradation?

Break one dependency and check that the main flow still works and the result says what was degraded.

In the project: `tc152`: the carrier is down, the sale goes through, shipping is marked as estimated.

Q41. What should happen to a broken `200`?

A success status isn't proof of success. Parse the body, check the fields you depend on, and treat anything else as a failure of that kind.

In the project: `tc115` and `tc145`.

Q42. Are scenarios safe with parallel tests?

Not on a shared server. A scenario's state is shared, so two tests can advance each other's state. Use one server per parallel test, or separate scenario names and matching on a per-test value.

4. Verification and test design

Q43. How do you verify a request was sent?

`verify(count, postRequestedFor(...))`, or read the requests with `findAll` and assert with your own library.

In the project: `verifyChargedExactly`, and `chargeCount` with AssertJ.

Q44. How do you verify something was not called?

`verify(0, ...)`, or assert that the list of requests is empty.

In the project: `tc155`: an invalid order reaches neither API.

Q45. What happens with an unmatched request?

WireMock answers `404` and logs it. Your code may treat that as a normal failure, so a test can pass by accident. Turn on `failOnUnmatchedRequests` so that any unmatched request fails the test.

In the project: `MockServers.strict`.

Q46. How do you debug a stub that doesn't match?

Read the diff in the `404` body, or use near misses (`findNearMissesForAllUnmatchedRequests`), which show the closest stubs and what differed. The usual causes are a header's case, a trailing slash, or a query string in `urlPathEqualTo`.

Q47. What is the page object of API mocking?

A stub class per API with methods named after behaviour. Tests read `provider.declines("card_expired")` and the paths, JSON and WireMock calls stay in one place.

In the project: `PaymentProviderStub` and `ShippingCarrierStub`.

Q48. Why not share path constants between client and stub?

Because a typo would then pass on both sides. Writing the path twice is a small cost for an independent check.

Q49. How do you know your mocks' tests can really fail?

Break the code on purpose and check that a test goes red. Mutation testing automates this.

In the project: eight deliberate breaks. One survived, because WireMock decodes `+` and `%20` alike, and `tc142` was tightened to assert the raw URL. See section 9 of `docs/framework-design.md`.

Q50. How do you keep mock-based tests from going stale?

Contract tests, a scheduled run against the real sandbox, and re-recording stubs now and then. A mock with no link to reality slowly turns into a test of your own assumptions.

Q51. How do you handle test data in stubs?

A builder with valid defaults, so a test states only the value it's about. Keep real personal data and real secrets out of stubs.

In the project: `CheckoutRequests.anOrder()`.

Q52. How do you structure a test on a mock?

Given the stub's behaviour, when the code runs, then assert the result and, where it matters, the request. Test one behaviour per test, and name it after that behaviour.

In the project: the step comments and the `tcNNN_` method names.

5. Process and scenarios

Q53. What is record and replay?

WireMock proxies to the real API, keeps what it sees as stubs, and answers alone afterwards. It's a fast way to get realistic stubs.

In the project: `RecordAndReplayTest`, with a second WireMock as the provider.

Q54. What are the risks of recording?

Secrets and personal data ending up in stubs, stubs that are too strict (for example matching a timestamp), and recordings going stale. Review them before committing, and filter the headers you capture.

Q55. Your stubs pass, but production fails. What do you check?

Compare a real request and response with the stub: field names, casing, encoding, headers, status codes. Then ask why nothing caught the difference, and add a contract test or a sandbox check for it.

Q56. A team wants a shared mock for a UI and mobile app. What do you set up?

Standalone WireMock or Docker, with JSON mappings in version control, one owner, and a way to choose a scenario per test, for example a header or a magic value such as a card token.

In the project: the `tok_declined` mapping shows the magic value idea.

Q57. How would you test a provider's webhook?

Two directions: stub the provider's API, and send the webhook to your own endpoint, including duplicates, out-of-order events and a bad signature.

Q58. How do you run these tests in CI?

Like any JUnit suite. WireMock starts inside the tests on a free port, so a plain runner is enough, and no service containers are needed.

In the project: `.github/workflows/ci.yml` runs `./mvnw verify`.

Q59. WireMock, MockServer, Mockoon or Hoverfly?

All are valid. WireMock has the broadest Java use and fits JUnit well. MockServer is similar. Mockoon has a desktop UI, which suits quick local mocks. Hoverfly is proxy-based and strong in capture and simulate. I'd choose by team skills and how the mock will be run.

Q60. How would you explain the value of this to a manager?

We can test what happens when a payment provider fails, on every commit, in seconds and without a sandbox. Those failure paths are the ones that cost money or double-charge customers, and we find problems before release.